

upstream_proposal_20260926

BOB-191 — upstream proposal: language arms for the constitution fail-open scanner

Revision: 1 **Last modified:** 2026-09-26T14:45:00Z

Status

Proposal only. Nothing in constitution/ was edited (canonical layer; this stream is not authorised to change it). The arms below run today as a consumer-side, advisory stand-in: scripts/pre_build/check_fail_open_unanalysed_langs.sh, called from pre-build invariant 39.

Problem (measured)

constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh analyses Python (ast) and C-family catch. For .go/.rs/.rb/.c it reports NOTE – N file(s) of an UNANALYSED extension and PASS (PARTIAL COVERAGE). Reproduced 2026-09-26 on a two-file fixture (an empty if err != nil {} in Go and an empty if (rc != 0) {} in C): the scanner exited 0 with the UNANALYSED note; the consumer arm reported both lines (svc.go:7 GO-EMPTY-ERR-BRANCH, a.c:3 C-EMPTY-ERR-BRANCH) and exited 1.

Proposed change to the constitution scanner

1. Add `scripts/gates/lib/failopen_go/` (stdlib-only Go program, `go/ast`) with two rules, `GO-EMPTY-ERR-BRANCH` and `GO-SWALLOWED-PANIC` — copy of `boba/scripts/pre_build/failopen_go/main.go`.
2. Add `scripts/gates/lib/failopen_lang_analyzer.py` with the Rust, Ruby and C rules — copy of `boba/scripts/pre_build/failopen_lang_analyzer.py`, including its `--selfcheck` golden-bad and golden-good snippets.
3. In the scanner, dispatch `.go` to the Go analyser and `.rs/.rb/.c/.h` to the Python analyser; run each analyser's control needle before scanning and exit 2 if a needle is not seen; remove a language from `DANGEROUS_COMBO_UNANALYSED_EXT` only when its analyser actually ran; keep a missing toolchain reported as `UNANALYSED`.
4. Carry the consumer's 15-arm test (per-language golden-bad and golden-good, the real `auth_middleware.go` golden-false fixture, comment/string golden-false, two paired mutations) into the constitution's mutation suite.

Once it lands, boba removes its stand-in and the invariant-39 arm (§11.4.28: the canonical copy lives in one place).

Honest boundary

- Only shapes fail-open by construction are reported. `if err != nil { return false }` is fail-closed and is not flagged (the §11.4.250 prediction recorded on BOB-191).
- A Go error discarded through the blank identifier needs type information and is not detected; that needs a type-aware tool (`errcheck` or `staticcheck`).
- On boba's real roots today: 0 hits across 122 Go files and 0 Rust/Ruby/C files. That zero covers only the two Go idioms above; it is not a claim that the Go code has no fail-open behaviour (BOB-204's discarded delete error is a different idiom and is not seen).